API Reference

Copy for LLM View as Markdown Install skills

The Stripe API is organized around REST. Our API has predictable resource-oriented URLs, accepts form-encoded request bodies, returns JSON-encoded responses, and uses standard HTTP response codes, authentication, and verbs.

You can use the Stripe API in sandboxes without affecting your live data or interacting with banking networks. The API key that you use to authenticate the request determines whether the request runs in live mode or in a sandbox. Sandboxes support all v2 APIs. Test mode sandboxes support some v2 APIs.

The Stripe API doesn't support bulk updates. You can work on only one object per request.

The Stripe API differs for every account as we release new versions and tailor functionality. Log in to see docs with your test key and data.

Was this section helpful? Yes No

Just getting started?

Check out our development quickstart guide.

Not a developer?

Use Stripe's no-code options or apps from our partners to get started with Stripe and to do more with your Stripe account—no code required.

BASE URL

```
https://api.stripe.com
```

CLIENT LIBRARIES

Ruby Python PHP Java Node.js Go .NET

By default, the Stripe API Docs demonstrate using curl to interact with the API over HTTP. Select one of our official client libraries to see examples in code.

Authentication

Copy for LLM View as Markdown Install skills

The Stripe API uses API keys to authenticate requests. You can view and manage your API keys in the Stripe Dashboard.

Test secret keys have the prefix `sk_test_` and live mode secret keys have the prefix `sk_live_`. Alternatively, you can use restricted API keys for granular permissions.

Your API keys carry many privileges, so be sure to keep them secure! Do not share your secret API keys in publicly accessible areas such as GitHub, client-side code, and so forth.

All API requests must be made over HTTPS. Calls made over plain HTTP will fail. API requests without authentication will also fail.

Was this section helpful? Yes No

AUTHENTICATED REQUEST cURL

```
curl https://api.stripe.com/v1/charges \
  -u sk_test_tR3PYbc...96tH88S4VQ2u:
# The colon prevents curl from asking for a password.
```

YOUR API KEY

A sample test API key is included in all the examples here, so you can test any example right away. Do not submit any personally identifiable information in requests made with this key.

To test requests using your account, replace the sample API key with your actual API key or sign in.

Errors

Copy for LLM View as Markdown Install skills

Stripe uses conventional HTTP response codes to indicate the success or failure of an API request. In general: Codes in the `2xx` range indicate success. Codes in the `4xx` range indicate an error that failed given the information provided (e.g., a required parameter was omitted, a charge failed, etc.). Codes in the `5xx` range indicate an error with Stripe's servers (these are rare).

Some `4xx` errors that could be handled programmatically (e.g., a card is declined) include an error code that briefly explains the error reported.

Was this section helpful? Yes No

Attributes

code nullable string

For some errors that could be handled programmatically, a short string indicating the error code reported.

decline_code nullable string

For card errors resulting from a card issuer decline, a short string indicating the card issuer's reason for the decline if they provide one.

message nullable string

A human-readable message providing more details about the error. For card errors, these messages can be shown to your users.

param nullable string

If the error is parameter-specific, the parameter related to the error. For example, you can use this to display a message near the correct form field.

payment_intent nullable object

The PaymentIntent object for errors returned on a request involving a PaymentIntent.

type enum

The type of error returned. One of `api_error`, `card_error`, `idempotency_error`, or `invalid_request_error`

Possible enum values
`api_error`
`card_error`
`idempotency_error`
`invalid_request_error`

More

Expand all

advice_code nullable string

charge nullable string

doc_url nullable string

network_advice_code nullable string

network_decline_code nullable string

payment_method nullable object

payment_method_type nullable string

request_log_url nullable string

setup_intent nullable object

source nullable object

HTTP STATUS CODE SUMMARY

200	OK	Everything worked as expected.
400	Bad Request	The request was unacceptable, often due to missing a required parameter.
401	Unauthorized	No valid API key provided.
402	Request Failed	The parameters were valid but the request failed.
403	Forbidden	The API key doesn't have permissions to perform the request.
404	Not Found	The requested resource doesn't exist.
409	Conflict	The request conflicts with another request (perhaps due to using the same idempotent key).
424	External Dependency Failed	The request couldn't be completed due to a failure in a dependency external to Stripe.
429	Too Many Requests	Too many requests hit the API too quickly. We recommend an exponential backoff of your requests.
500, 502, 503, 504	Server Errors	Something went wrong on Stripe's end. (These are rare.)

ERROR TYPES

`api_error`	API errors cover any other type of problem (e.g., a temporary problem with Stripe's servers), and are extremely uncommon.

`card_error`	Card errors are the most common type of error you should expect to handle. They result when the user enters a card that can't be charged for some reason.
`idempotency_error`	Idempotency errors occur when an `Idempotency-Key` is re-used on a request that does not match the first request's API endpoint and parameters.
`invalid_request_error`	Invalid request errors arise when your request has invalid parameters.

Handling errors

Copy for LLM View as Markdown Install skills

Our Client libraries raise exceptions for many reasons, such as a failed charge, invalid parameters, authentication errors, and network unavailability. We recommend writing code that gracefully handles all possible API exceptions.

Related guide: Error Handling

cURL

```
# Select a client library to see examples of
# handling different kinds of errors.
```

Expanding Responses

Copy for LLM View as Markdown Install skills

Many objects allow you to request additional information as an expanded response by using the `expand` request parameter. This parameter is available on all API requests, and applies to the response of that request only. You can expand responses in two ways.

In many cases, an object contains the ID of a related object in its response properties. For example, a `Charge` might have an associated Customer ID. You can expand these objects in line with the expand request parameter. The `expandable` label in this documentation indicates ID fields that you can expand into objects.

Some available fields aren't included in the responses by default, such as the `number` and `cvc` fields for the Issuing Card object. You can request these fields as an expanded response by using the `expand` request parameter.

You can expand recursively by specifying nested fields after a dot (`.`). For example, requesting `payment_intent.customer` on a charge expands the `payment_intent` property into a full

PaymentIntent object, then expands the `customer` property on that payment intent into a full Customer object.

You can use the `expand` parameter on any endpoint that returns expandable fields, including list, create, and update endpoints.

Expansions on list requests start with the `data` property. For example, you can expand `data.customers` on a request to list charges and associated customers. Performing deep expansions on numerous list requests might result in slower processing times.

Expansions have a maximum depth of four levels (for example, the deepest expansion allowed when listing charges is `data.payment_intent.customer.default_source`).

You can expand multiple objects at the same time by identifying multiple items in the `expand` array.

Related guide: Expanding responses
Related video: Expand

Was this section helpful? Yes No

cURL

```
curl https://api.stripe.com/v1/charges/ch_3LmzzQ2eZvKYlo2C0XjzUzJV \
  -u sk_test_tR3PYbc...96tH88S4VQ2u: \
  -d "expand[]"=customer \
  -d "expand[]"="payment_intent.customer" \
  -G
```

RESPONSE


```
{
  "id": "ch_3LmzzQ2eZvKYlo2C0XjzUzJV",
  "object": "charge",
  "customer": {
    "id": "cu_14HOpH2eZvKYlo2CxXIM7Pb2",
    "object": "customer",
    // ...
  },
  "payment_intent": {
    "id": "pi_3MtwBwLkdIwHu7ix28a3tqPa",
    "object": "payment_intent",
    "customer": {
      "id": "cus_NffrFeUfNV2Hib",
      "object": "customer",
      // ...
    },
    // ...
  },
  // ...
}
```

Idempotent requests

Copy for LLM View as Markdown Install skills

The API supports idempotency for safely retrying requests without accidentally performing the same operation twice. When creating or updating an object, use an idempotency key. Then, if a connection error occurs, you can safely repeat the request without risk of creating a second object or performing the update twice.

To perform an idempotent request, provide an additional `IdempotencyKey` element to the request options.

Stripe's idempotency works by saving the resulting status code and body of the first request made for any given idempotency key, regardless of whether it succeeds or fails. Subsequent requests with the same key return the same result, including `500` errors.

A client generates an idempotency key, which is a unique key that the server uses to recognize subsequent retries of the same request. How you create unique keys is up to you, but we suggest using V4 UUIDs, or another random string with enough entropy to avoid collisions. Idempotency keys are up to 255 characters long. Avoid using sensitive data (for example, email addresses or personal identifiers) as idempotency keys.

You can remove keys from the system automatically after they're at least 24 hours old. We generate a new request if a key is reused after the original is pruned. The idempotency layer compares incoming parameters to those of the original request and errors if they're not the same to prevent accidental misuse.

We save results only after the execution of an endpoint begins. If incoming parameters fail validation, or the request conflicts with another request that's executing concurrently, we don't save the idempotent result because no API endpoint initiates the execution. You can retry these requests. Learn more about when you can retry idempotent requests.

All `POST` requests accept idempotency keys. Don't send idempotency keys in `GET` and `DELETE` requests because it has no effect. These requests are idempotent by definition.

Was this section helpful? Yes No

cURL

```
curl https://api.stripe.com/v1/customers \
  -u sk_test_tR3PYbc...96tH88S4VQ2u: \
  -H "Idempotency-Key: KG5LxwFBepaKHyUD" \
  -d description="My First Test Customer (created for API docs at https://
```

Include-dependent response values (API v2)

Copy for LLM View as Markdown Install skills

Some API v2 responses contain null values for certain properties by default, regardless of their actual values. That reduces the size of response payloads while maintaining the basic response structure. To retrieve the actual values for those properties, specify them in the `include` array request parameter.

To determine whether you need to use the `include` parameter in a given request, look at the request description. The `include` parameter's enum values represent the response properties that depend on the `include` parameter.

NOTE

Whether a response property defaults to null depends on the request endpoint, not the object that the endpoint references. If multiple endpoints return data from the same object, a particular property can depend on `include` in one endpoint and return its actual value by default for a different endpoint.

A hash property can depend on a single `include` value, or on multiple `include` values associated with its child properties. For example, when updating an Account, to return actual values for the entire `identity` hash, specify `identity` in the `include` parameter. Otherwise, the `identity` hash is null in the response. However, to return actual values for the `configuration` hash, you must specify individual configurations in the request. If you specify at least one configuration, but not all of them, specified configurations return actual values and unspecified configurations return null. If you don't specify any configurations, the `configuration` hash is null in the response.

Related guide: Include-dependent response values

POST /v2/core/accounts cURL

```
curl -X POST https://api.stripe.com/v2/core/accounts \
  -H "Authorization: Bearer sk_test_tR3PYbc...96tH88S4VQ2u" \
  -H "Stripe-Version: 2026-04-22.preview" \
  --json '{
    "include": [
        "identity",
        "configuration.customer"
    ]
  }'
```

INCLUDED RESPONSE PROPERTIES

The response includes actual values for the properties specified in the `include` parameter, and null for all other include-dependent properties.

RESPONSE


```
{
  "id": "acct_123",
  "object": "v2.core.account",
  "applied_configurations": [
    "customer",
    "merchant"
  ],
  "configuration": {
    "customer": {
      "automatic_indirect_tax": {
        ...
      },
      "billing": {
        ...
      },
      "capabilities": {
        ...
      },
      ...
    },
    "merchant": null,
    "recipient": null
  },
  "contact_email": "furever@example.com",
  "created": "2025-06-09T21:16:03.000Z",
  "dashboard": "full",
  "defaults": null,
  "display_name": "Furever",
  "identity": {
    "business_details": {
```


Metadata

Copy for LLM View as Markdown Install skills

Updateable Stripe objects—including Account, Charge, Customer, PaymentIntent, Refund, Subscription, and Transfer have a `metadata` parameter. You can use this parameter to attach key-value data to these Stripe objects.

You can specify up to 50 keys, with key names up to 40 characters long and values up to 500 characters long. Keys and values are stored as strings and can contain any characters with one exception: you can't use square brackets ([and]) in keys.

You can use metadata to store additional, structured information on an object. For example, you could store your user's full name and corresponding unique identifier from your system on a Stripe Customer object. Stripe doesn't use metadata—for example, we don't use it to authorize or decline a charge and it won't be seen by your users unless you choose to show it to them.

Some of the objects listed above also support a `description` parameter. You can use the `description` parameter to annotate a charge-for example, a human-readable description such as `2 shirts for test@example.com`. Unlike `metadata`, `description` is a single string, which your users might see (for example, in email receipts Stripe sends on your behalf).

Don't store any sensitive information (bank account numbers, card details, and so on) as metadata or in the `description` parameter.

Related guide: Metadata

Sample metadata use cases

- **Link IDs**: Attach your system's unique IDs to a Stripe object to simplify lookups. For example, add your order number to a charge, your user ID to a customer or recipient, or a unique receipt number to a transfer.
- **Refund papertrails**: Store information about the reason for a refund and the individual responsible for its creation.
- **Customer details**: Annotate a customer by storing an internal ID for your future use.

Was this section helpful? Yes No

POST /v1/customers cURL

```
curl https://api.stripe.com/v1/customers \
  -u "sk_test_tR3PYbc...96tH88S4VQ2u:" \
  -d "metadata[order_id]=6735"
```


```
{
  "id": "cus_123456789",
  "object": "customer",
  "address": {
    "city": "city",
    "country": "US",
    "line1": "line 1",
    "line2": "line 2",
    "postal_code": "90210",
    "state": "CA"
  },
  "balance": 0,
  "created": 1483565364,
  "currency": null,
  "default_source": null,
  "delinquent": false,
  "description": null,
  "discount": null,
  "email": null,
  "invoice_prefix": "C11F7E1",
  "invoice_settings": {
```

```
    "custom_fields": null,
    "default_payment_method": null,
    "footer": null,
    "rendering_options": null
  },
  "livemode": false,
  "metadata": {
    "order_id": "6735"
  },
  "name": null,
  "next_invoice_sequence": 1,
  "phone": null,
  "preferred_locales": [],
  "shipping": null,
  "tax_exempt": "none"
}
```


Pagination

Copy for LLM View as Markdown Install skills

All top-level API resources have support for bulk fetches through “list” API methods. For example, you can list charges, list customers, and list invoices. These list API methods share a common structure and accept, at a minimum, the following three parameters: `limit`, `starting_after`, and `ending_before`.

Stripe’s list API methods use cursor-based pagination through the `starting_after` and `ending_before` parameters. Both parameters accept an existing object ID value (see below) and return objects in reverse chronological order. The `ending_before` parameter returns objects listed before the named object. The `starting_after` parameter returns objects listed after the named object. These parameters are mutually exclusive. You can use either the `starting_after` or `ending_before` parameter, but not both simultaneously.

Our client libraries offer auto-pagination helpers to traverse all pages of a list.

Parameters

`limit` optional, default is 10

This specifies a limit on the number of objects to return, ranging between 1 and 100.

`starting_after` optional object ID

A cursor to use in pagination. `starting_after` is an object ID that defines your place in the list. For example, if you make a list request and receive 100 objects, ending with `obj_foo`, your subsequent call can include `starting_after=obj_foo` to fetch the next page of the list.

ending_before optional object ID

A cursor to use in pagination. `ending_before` is an object ID that defines your place in the list. For example, if you make a list request and receive 100 objects, starting with `obj_bar`, your subsequent call can include `ending_before=obj_bar` to fetch the previous page of the list.

List Response Format

object string, value is "list"

A string that provides a description of the object type that returns.

data array

An array containing the actual response elements, paginated by any request parameters.

has_more boolean

Whether or not there are more elements available after this set. If `false`, this set comprises the end of the list.

url url

The URL for accessing this list.

V2 API PAGINATION

APIs within the `/v2` namespace contain a different pagination interface than the `v1` namespace.

Was this section helpful? Yes No

RESPONSE


```
{
  "object": "list",
  "url": "/v1/customers",
  "has_more": false,
  "data": [
    {
      "id": "cus_4QFJOjw2pOmAGJ",
      "object": "customer",
      "address": null,
      "balance": 0,
      "created": 1405641735,
      "currency": "usd",
      "default_source": "card_14HOpG2eZvKYlo2Cz4u5AJG5",
      "delinquent": false,
      "description": "New customer",
      "discount": null,
      "email": null,
```

```
      "invoice_prefix": "7D11B54",
      "invoice_settings": {
        "custom_fields": null,
        "default_payment_method": null,
        "footer": null,
        "rendering_options": null
      },
      "livemode": false,
      "metadata": {
        "order_id": "6735"
      },
      "name": "cus_4QFJOjw2pOmAGJ",
      "next_invoice_sequence": 25,
      "phone": null,
      "preferred_locales": [],
      "shipping": null,
      "tax_exempt": "none",
      "test_clock": null
    },
  ]
}
```


Search

Copy for LLM View as Markdown Install skills

Some top-level API resource have support for retrieval via “search” API methods. For example, you can search charges, search customers, and search subscriptions.

Stripe’s search API methods utilize cursor-based pagination via the `page` request parameter and `next_page` response parameter. For example, if you make a search request and receive `"next_page": "pagination_key"` in the response, your subsequent call can include `page=pagination_key` to fetch the next page of results.

Our client libraries offer auto-pagination helpers to easily traverse all pages of a search result.

Search request format

`query` required

The search query string. See search query language.

`limit` optional

A limit on the number of objects returned. Limit can range between 1 and 100, and the default is 10.

`page` optional

A cursor for pagination across multiple pages of results. Don't include this parameter on the first call. Use the `next_page` value returned in a previous response to request subsequent results.

Search response format

`object` string, value is "search_result"

A string describing the object type returned.

`url` string

The URL for accessing this list.

`has_more` boolean

Whether or not there are more elements available after this set. If `false`, this set comprises the end of the list.

`data` array

An array containing the actual response elements, paginated by any request parameters.

`next_page` string

A cursor for use in pagination. If `has_more` is true, you can pass the value of `next_page` to a subsequent call to fetch the next page of results.

`total_count` optional positive integer or zero

The total number of objects that match the query, only accurate up to 10,000. This field isn't included by default. To include it in the response, expand the `total_count` field.

Was this section helpful? Yes No

RESPONSE


```
{
  "object": "search_result",
  "url": "/v1/customers/search",
  "has_more": false,
  "data": [
    {
      "id": "cus_4QFJOjw2pOmAGJ",
      "object": "customer",
      "address": null,
      "balance": 0,
      "created": 1405641735,
      "currency": "usd",
      "default_source": "card_14HOpG2eZvKYlo2Cz4u5AJG5",
      "delinquent": false,
```

```
      "description": "someone@example.com for Coderwall",
      "discount": null,
      "email": null,
      "invoice_prefix": "7D11B54",
      "invoice_settings": {
        "custom_fields": null,
        "default_payment_method": null,
        "footer": null,
        "rendering_options": null
      },
      "livemode": false,
      "metadata": {
        "foo": "bar"
      },
      "name": "fakename",
      "next_invoice_sequence": 25,
      "phone": null,
      "preferred_locales": [],
      "shipping": null,
      "tax_exempt": "none",
      "test_clock": null
    }
  ]
}
```


Auto-pagination

Copy for LLM View as Markdown Install skills

Our libraries support auto-pagination. This feature allows you to easily iterate through large lists of resources without having to manually perform the requests to fetch subsequent pages.

Was this section helpful? Yes No

cURL

```
# The auto-pagination feature is specific to Stripe's
# libraries and cannot be used directly with curl.
```

Request IDs

Copy for LLM View as Markdown Install skills

Each API request has an associated request identifier. You can find this value in the response headers, under `Request-Id`. You can also find request identifiers in the URLs of individual request logs in your Dashboard.

To expedite the resolution process, provide the request identifier when you contact us about a specific request.

Was this section helpful? Yes No

cURL

```
curl https://api.stripe.com/v1/customers \
  -u sk_test_tR3PYbc...96tH88S4VQ2u: \
  -D "-" \
  -X POST
```

Connected Accounts

Copy for LLM View as Markdown Install skills

If you use Stripe Connect, you can issue requests on behalf of your connected accounts. To act as a connected account, include a `Stripe-Account` header containing the connected account ID, which typically starts with the `acct_` prefix.

The connected account ID is set per-request. Methods on the returned object reuse the same account ID.

Related guide: Making API calls for connected accounts

Was this section helpful? Yes No

PER-REQUEST ACCOUNT cURL

```
curl https://api.stripe.com/v1/charges/ch_3LmjFA2eZvKYlo2C09TLIsrw \
  -u sk_test_tR3PYbc...96tH88S4VQ2u: \
  -H "Stripe-Account: acct_1032D82eZvKYlo2C" \
  -G
```

Versioning

Copy for LLM View as Markdown Install skills

Each major release, such as Acacia, includes changes that aren't backward-compatible with previous releases. Upgrading to a new major release can require updates to existing code. Each monthly release includes only backward-compatible changes, and uses the same name as the last major release. You can safely upgrade to a new monthly release without breaking any existing code. The current version is 2026-04-22.dahlia. For information on all API versions, view our API changelog.

You can upgrade your API version in Workbench. As a precaution, use API versioning to test a new API version before committing to an upgrade.

Was this section helpful? Yes No